DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT – V : Transactions, Concurrency, Recovery & Indexing

Easy handwritten notes with diagrams for students

Topics in this unit

1. Transaction concept, states, ACID properties
2. Concurrent executions, serializability, recoverability
3. Implementation of isolation, testing for serializability
4. Concurrency protocols: lock based, timestamp based, optimistic
5. Deadlocks
6. Failure classification, storage, recovery and atomicity, recovery algorithm
7. Indexing: B+ trees and operations
8. Hash based indexing

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm. Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing.

Part I : Transaction Concept

1.1 What is a Transaction?

Definition: A **transaction** is a **logical unit of work**: a sequence of read / write operations on the database that must be executed **completely or not at all**. It starts with BEGIN and ends with COMMIT or ROLLBACK.

Example - transfer Rs. 500 from account A to account B:

```
T1:  read(A);       A = A - 500;   write(A);
     read(B);       B = B + 500;   write(B);
     commit;
```

If the system fails after **write(A)** but before **write(B)**, money disappears. The transaction concept prevents this.

Basic operations: **read(X)** copies X from the database to a local buffer; **write(X)** writes the local value back to the database; **commit** makes changes permanent; **rollback / abort** undoes all changes.

1.2 ACID Properties

Property	Meaning	Ensured by	Example
Atomicity	All operations happen or none (all-or-nothing)	Recovery manager (log, undo)	Debit and credit both done or both undone
Consistency	DB moves from one valid state to another valid state; constraints hold	Programmer + integrity constraints	Total A + B same before and after
Isolation	Concurrent transactions do not interfere; appears as if run one by one	Concurrency-control manager (locks)	T2 cannot see T1's half-done data
Durability	Once committed, changes survive any crash	Recovery manager (log on stable storage)	Committed transfer remains after power failure

1.3 Transaction States

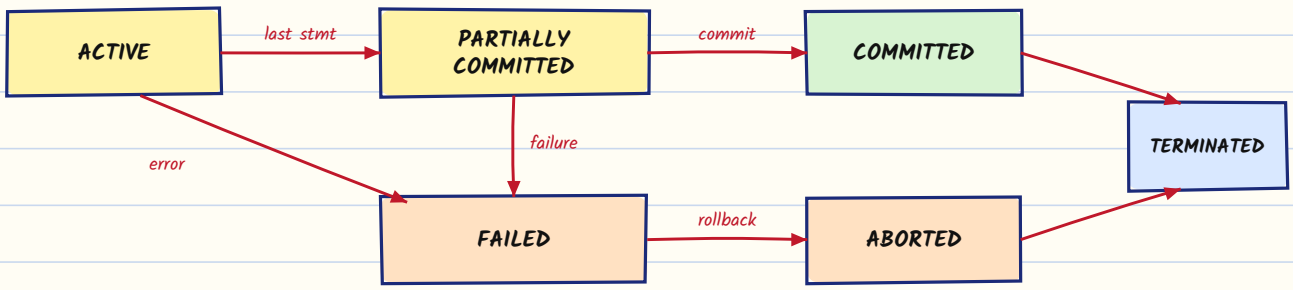


Fig 5.1 : Transaction state diagram

State	Meaning
Active	Initial state; transaction is executing its operations
Partially committed	After the final statement; results may still be only in buffer
Committed	Changes are safely written to the database - permanent

State	Meaning
Failed	Normal execution cannot continue (error / crash)
Aborted	Changes rolled back, DB restored to state before the transaction. Then restart (if no logical error) or kill
Terminated	Transaction has left the system

Part 2 : Concurrent Executions

Running many transactions at the same time (interleaving their operations) gives higher **throughput**, better **resource utilisation** and lower **waiting time**, but may produce wrong results if not controlled. A **schedule** is the order in which operations of transactions are executed.

2.1 Problems of Uncontrolled Concurrency

Problem	What happens	Example
Lost update (W-W)	Two transactions update the same item; one overwrites the other	T1 and T2 both read A=100; T1 writes 90, T2 writes 120 -> T1's update is lost
Dirty read (W-R)	T2 reads a value written by T1 that is not yet committed; T1 then aborts	T2 used data that never really existed
Unrepeatable read (R-W)	T1 reads X twice and gets different values because T2 changed X in between	Price shows 50 then 60 in same transaction
Phantom read	A repeated range query returns new rows inserted by another transaction	COUNT(*) is 10, later 11

2.2 Serial and Non-serial Schedules

- **Serial schedule** - transactions run one after another with no interleaving. Always correct (n! possible orders) but slow.
- **Non-serial schedule** - operations interleaved; fast but must be equivalent to some serial schedule to be correct.
- A non-serial schedule that is equivalent to a serial schedule is **serializable**.

2.3 Serializability

(a) Conflict serializability

Two operations **conflict** if they belong to different transactions, access the **same data item** and at least one is a **write**.

T1	T2	Conflict?
read(X)	read(X)	No
read(X)	write(X)	Yes (R-W)
write(X)	read(X)	Yes (W-R)
write(X)	write(X)	Yes (W-W)
any on X	any on Y (different item)	No

A schedule is **conflict serializable** if it can be converted to a serial schedule by swapping adjacent **non-conflicting** operations.

(b) View serializability

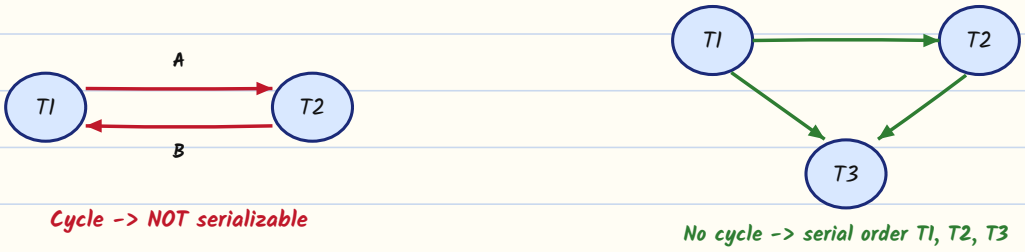
Schedules S and S' are **view equivalent** if: (1) each transaction reads the same initial value of an item in both, (2) each read gets its value from the same writer in both, (3) the final write of every item is by the same transaction. Every conflict-serializable schedule is view-serializable, but not the reverse (view serializable schedules containing **blind writes** may not be conflict serializable). Testing view serializability is NP-complete.

2.4 Testing for Serializability – Precedence Graph

Build a **precedence (serialization) graph**: one node per transaction; draw edge Ti -> Tj if an operation of Ti conflicts with a LATER operation of Tj. **If the graph has a cycle, the schedule is NOT conflict serializable; if it has no cycle, it is serializable**, and a topological order of the graph gives the equivalent serial order.

Schedule S1	T1	T2
1	read(A); write(A)	
2		read(A); write(A)
3		read(B); write(B)
4	read(B); write(B)	

Item A: T1 writes before T2 touches it -> edge T1 -> T2. Item B: T2 writes before T1 touches it -> edge T2 -> T1. Result: a **cycle**.



Cycle -> NOT serializable

No cycle -> serial order T1, T2, T3

Fig S.2 : Precedence graphs – left has a cycle, right has none

2.5 Recoverability

A schedule must allow the system to recover correctly from failures. The key question: if T2 reads data written by T1, what if T1 aborts?

Schedule type	Rule	Property
Recoverable	If Tj reads data written by Ti, then Ti must commit **before** Tj commits	Avoids committed transaction having read uncommitted-then-aborted data
Cascadeless (ACA)	Tj may read an item only after the writer Ti has **committed**	No cascading rollback (one abort does not force others to abort)
Strict	No transaction reads OR overwrites an item written by Ti until Ti commits / aborts	Easy undo; used by most DBMS

Relationship: **Strict** ⊂ **Cascadeless** ⊂ **Recoverable**. A **cascading rollback** happens when abort of Ti forces abort of Tj that read Ti's dirty data, and so on.

2.6 Implementation of Isolation (Isolation levels)

Full isolation (serializability) reduces concurrency, so SQL defines weaker levels. A higher level prevents more anomalies but allows less concurrency.

Isolation level	Dirty read	Unrepeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Isolation is implemented using **locking** (2PL), **timestamps**, **optimistic validation** or **multiversion (snapshot) isolation**, explained next.

Part 3 : Concurrency Control Protocols

3.1 Lock-Based Protocols

A **lock** is a variable attached to a data item that controls access to it. A transaction must obtain a lock before using the item.

Lock mode	Meaning	Operation
Shared (S)	Many transactions can hold it together	Read only: lock-S(X)
Exclusive (X)	Only one transaction can hold it	Read and write: lock-X(X)

Requested \ Held	S	X
S	Compatible (YES)	NO
X	NO	NO

Lock compatibility matrix

Release with unlock(X). Using locks alone is not enough: if T locks and unlocks carelessly, non-serializable results still occur. So we need a **locking protocol**.

Two-Phase Locking (2PL)

Each transaction acquires and releases locks in two phases:

- **Growing phase** - can obtain locks, cannot release any.
- **Shrinking phase** - can release locks, cannot obtain any new lock.
- The point where the last lock is obtained is the **lock point**. Transactions can be serialized in order of their lock points.

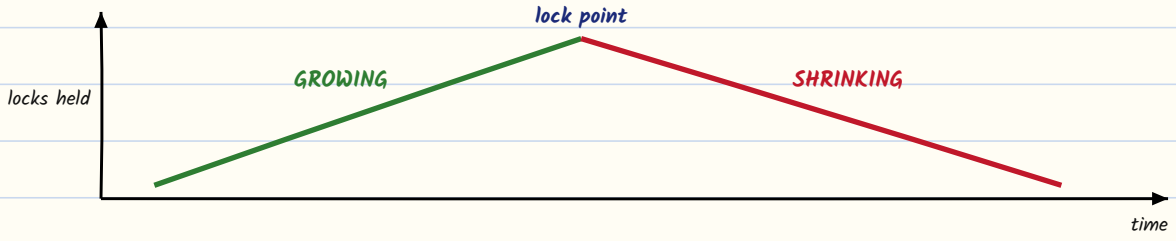


Fig 5.3 : Two-phase locking

Variant	Rule	Benefit
Basic 2PL	Growing then shrinking	Guarantees conflict serializability; may cause deadlock and cascading rollback

Variant	Rule	Benefit
Strict 2PL	Hold all **exclusive** locks until commit / abort	Cascadeless schedules
Rigorous 2PL	Hold **all** locks (S and X) until commit / abort	Serial order = commit order
Conservative 2PL	Lock all items before starting	Deadlock-free but hard to know items in advance

Lock conversion: upgrade S -> X in growing phase; downgrade X -> S in shrinking phase. Problems of locking: **deadlock** and **starvation**.

3.2 Timestamp-Based Protocols

Each transaction T gets a unique **timestamp TS(T)** when it starts (older = smaller). Each item Q keeps **R-TS(Q)** (largest TS that read Q) and **W-TS(Q)** (largest TS that wrote Q). The protocol makes the schedule equivalent to the serial order of timestamps. No locks, so **no deadlock**.

Operation by T on Q	Condition	Action
read(Q)	TS(T) < W-TS(Q) (Q already overwritten by a younger T)	Reject read, **rollback T** and restart with new timestamp
	TS(T) ≥ W-TS(Q)	Execute; R-TS(Q) = max(R-TS(Q), TS(T))
write(Q)	TS(T) < R-TS(Q) (a younger T already read Q)	Reject write, rollback T
	TS(T) < W-TS(Q) (a younger T already wrote Q)	Reject write, rollback T (Thomas' write rule: just **ignore** this obsolete write)
	Otherwise	Execute; W-TS(Q) = TS(T)

- Ensures conflict serializability and freedom from deadlock.
- Drawbacks: possible **starvation** of long transactions; schedules may not be recoverable (fix by delaying commits).

3.3 Optimistic (Validation-Based) Protocol

Assumes conflicts are **rare**, so transactions run without locks and are checked only at the end. Good for read-mostly workloads.

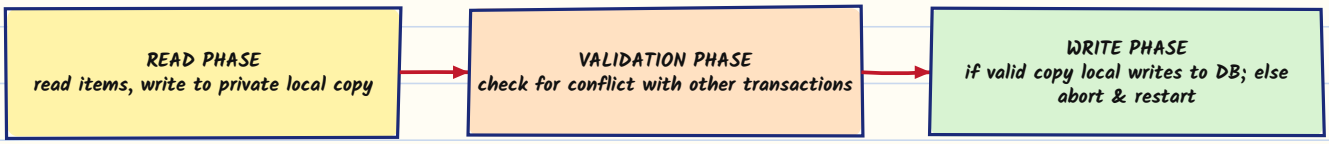


Fig 5.4 : Three phases of optimistic concurrency control

Validation test for Tj against every earlier transaction Ti (TS(Ti) < TS(Tj)): Tj passes if either (1) Ti finishes before Tj starts, or (2) Ti's write set does not intersect Tj's read set and Ti finishes its write phase before Tj starts its write phase. Otherwise Tj is **rolled back**.

* **Compare:** Locking = **pessimistic** (prevent conflicts, may wait / deadlock). Timestamp = order by age. Optimistic = **allow, then validate**, aborts on conflict. Multiversion (MVCC) keeps old versions so readers never block writers.

Part 4 : Deadlocks

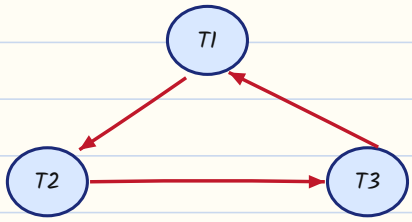
Definition: A **deadlock** is a situation in which two or more transactions are each waiting for a lock held by another, so **none can proceed**.

Example: T1 holds lock on A and requests B; T2 holds lock on B and requests A.

Four necessary (Coffman) conditions: **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**.

4.1 Deadlock Detection - Wait-for Graph

Node per transaction; edge Ti -> Tj if Ti is waiting for a lock held by Tj. **A cycle means deadlock**. The system checks the graph periodically.



Cycle T1 -> T2 -> T3 -> T1
= DEADLOCK

Fig 5.5 : Wait-for graph with a cycle

4.2 Deadlock Prevention

Scheme	If Ti requests a lock held by Tj	Type
Wait-Die	If Ti is **older** (smaller TS) it waits; if Ti is younger it **dies** (rolled back, restarts with same TS)	Non-preemptive
Wound-Wait	If Ti is **older** it **wounds** (aborts) Tj; if Ti is younger it waits	Preemptive
Timeout	Wait for fixed time, then roll back	Simple, may roll back needlessly
Conservative 2PL	Acquire all locks at the start	No hold-and-wait

4.3 Deadlock Recovery

- **Victim selection** - choose transaction with least cost (shortest running, fewest locks, least work done).
- **Rollback** - total (abort victim) or partial (back to a point that breaks the deadlock).
- Prevent **starvation** - include the number of rollbacks in the cost so the same victim is not always chosen.

Part 5 : Failures, Storage and Recovery

5.1 Failure Classification

Failure	Cause	Effect / recovery
Transaction failure	Logical error (bad input, overflow) or system error (deadlock)	Only that transaction aborted; rollback
System crash	Power cut, OS / hardware / software bug	Contents of main memory lost; disk safe. Restart recovery needed
Disk failure	Head crash, bad sectors, disk damage	Data on disk lost; recover from backup / archive + log

5.2 Storage Types

Storage	Survives system crash?	Examples
Volatile	No	Main memory, cache
Non-volatile	Yes (but disk can fail)	Disk, SSD, tape
Stable	Yes - never lost (theoretical); implemented by **replicating** data on several non-volatile devices	Mirrored disks, remote backup

Recovery must ensure atomicity and durability: updates of uncommitted transactions must be **undone** and updates of committed transactions must be **redone** if lost.

5.3 Log-Based Recovery (Atomicity and Durability)

The **log** is a sequence of records kept on stable storage. **Write-Ahead Logging (WAL)**: the log record must be written to stable storage **before** the corresponding data page is written to disk.

Log record	Meaning
<Ti, start>	Transaction Ti has started
<Ti, X, V1, V2>	Ti changed item X from old value V1 to new value V2
<Ti, commit>	Ti committed
<Ti, abort>	Ti aborted
<checkpoint L>	Checkpoint; L = list of active transactions

	Deferred database modification	Immediate database modification
Write to DB	Only after commit	Any time, even before commit
Log needs	New value only (redo)	Old and new value (undo + redo)
On crash	REDO committed transactions; ignore uncommitted	UNDO uncommitted, REDO committed

Recovery rule: after a crash scan the log; if it has both <Ti, start> and <Ti, commit> -> **redo(Ti)**; if it has <Ti, start> but no commit -> **undo(Ti)**.

Checkpoints

Scanning the whole log is slow. A **checkpoint** periodically forces all log records and modified buffer pages to disk and writes a checkpoint record. After a crash, only the transactions active at or after the last checkpoint need to be examined.

Shadow paging (alternative)

Keeps two page tables: **current** and **shadow**. Updates go to new pages via the current table; the shadow table is untouched. Commit = atomically make the current table the new shadow. Abort = discard the current table. No log is needed, but data gets fragmented and garbage must be collected.

5.4 Recovery Algorithm (ARIES)

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) is the standard algorithm. It uses WAL, **repeating history during redo** and logging of undo (CLR - compensation log records). Each log record has a **LSN** (log sequence number); each page stores PageLSN. Two tables are kept: **Dirty Page Table** and **Transaction Table**.

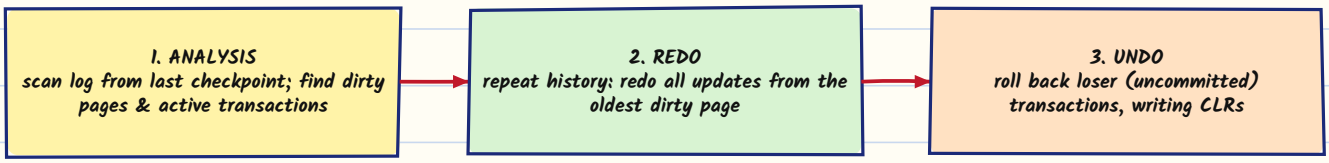


Fig 5.6 : Three passes of ARIES recovery

Pass	Work
Analysis	Starts at last checkpoint; rebuilds Dirty Page Table and Transaction Table; finds RedoLSN (where redo starts) and the loser transactions
Redo	Scans forward from RedoLSN; reapplies every logged update (committed or not) if page's PageLSN < record's LSN
Undo	Scans backward undoing updates of loser transactions; writes a CLR for each undo so work is never undone twice

Part 6 : Indexing Techniques

An **index** is an auxiliary data structure (like the index of a book) that lets the DBMS find rows by a **search key** without scanning the whole table. Indexes speed up reads but slow down inserts / updates and take space.

Index type	Meaning
Primary (clustering)	Index on the attribute by which the file is physically sorted
Secondary (non-clustering)	Index on another attribute; file is not sorted by it
Dense	One index entry for every record
Sparse	Index entry only for some records (one per block)
Ordered vs hash	Ordered: tree / sorted (B+ tree) - good for range queries. Hash: hash function - good for equality queries

6.1 B+ Tree

The **B+ tree** is a balanced multi-way search tree and the most common index structure (used in almost every RDBMS). All leaves are at the **same depth**, so every search costs the same: O(log n) disk accesses.

Properties (order n = maximum number of pointers in a node)

Node	Keys	Pointers / children
Root	at least 1 key	at least 2 children (unless it is a leaf)
Internal node	$\lceil (n-1)/2 \rceil$ to n-1	$\lceil n/2 \rceil$ to n children

Node	Keys	Pointers / children
Leaf node	$\lceil (n-1)/2 \rceil$ to n-1 keys + data pointers	last pointer goes to the next leaf

- Internal nodes store only **keys as guides**; **all actual data pointers are in the leaves**.
- Leaves are linked as a **sorted linked list** -> fast range queries and sequential scans.
- Difference from B tree: B tree stores data pointers in internal nodes too; B+ tree has more keys per internal node (shallower), faster range scan.

Example B+ tree of order 4 (max 3 keys, min 2 keys per leaf):

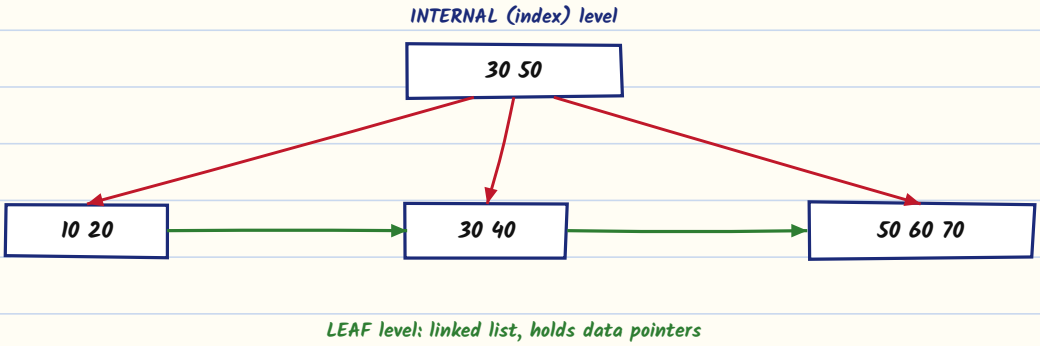


Fig 5.7 : B+ tree (order 4)

6.2 Operations on B+ Trees

Search

Start at the root; at each node follow the pointer for the key range containing the search key; at the leaf look for the key. For a range query, find the first key then follow the leaf links.

Insertion

- Find the correct leaf and insert the key in sorted order.
- If the leaf has room -> done.
- If the leaf **overflows** -> split it into two leaves; the **smallest key of the new right leaf is copied up** to the parent (leaf split: copy-up).
- If the parent overflows -> split it; the **middle key is moved up** (internal split: push-up). Splits may reach the root and increase tree height by one.

Example: insert 80 into the tree above. Leaf [50 60 70] becomes [50 60 70 80] (4 keys, overflow) -> split into [50 60] and [70 80]; key 70 is copied to the root:

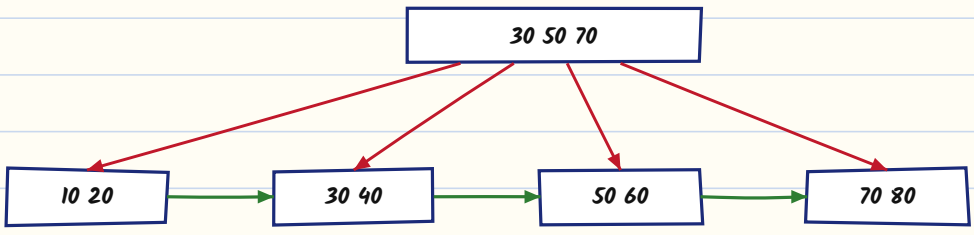


Fig 5.8 : B+ tree after inserting 80 (leaf split, 70 copied up)

Deletion

- Find the leaf and delete the key.

- If the leaf still has the minimum number of keys -> done (update parent key if the smallest key was removed).
- If it **underflows** -> first try to **borrow (redistribute)** a key from a sibling that has more than the minimum; update the parent separator.
- If the sibling cannot spare a key -> **merge** with the sibling and delete the separator key from the parent. Merges can propagate upward and may reduce the height.

Operation	Cost (disk accesses)
Search / insert / delete	$O(\log_f N)$, where f = fan-out (typically 100-500) -> 3 or 4 levels for millions of rows
Range query	Search first key + sequential leaf scan

6.3 Hash-Based Indexing

A **hash function** h(K) maps a search key directly to a **bucket** (a disk page holding records). Very fast for **equality search** (about 1 access) but **cannot do range queries** because order is lost.

Static hashing

- Number of buckets B is fixed. Bucket = h(K) = K mod B.
- Bucket full -> **overflow chain** of extra pages. Many overflows make search slow; too many empty buckets waste space; changing B needs full rehash.

Extendible hashing (dynamic)

Uses a **directory** of 2^d pointers (d = **global depth**) indexed by the first d bits of the hash value. Each bucket has a **local depth** d'. When a bucket is full it is **split**; if d' = d the directory is **doubled**. No overflow chains and no full rehash.

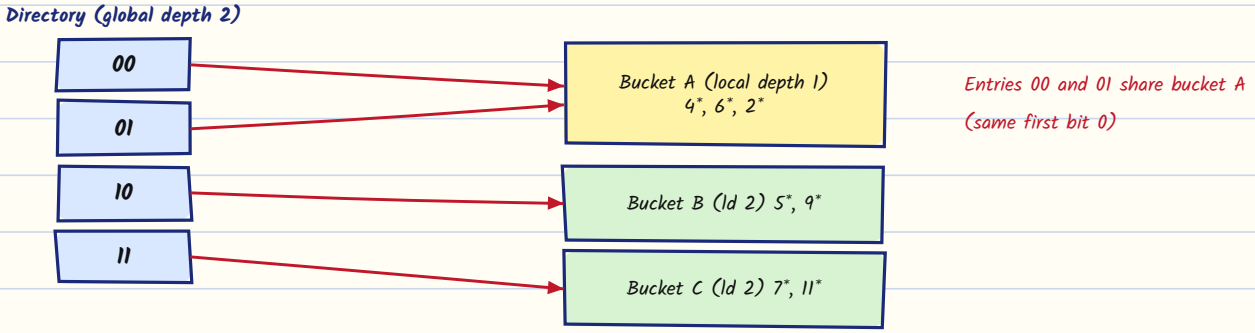


Fig 5.9 : Extendible hashing directory and buckets

Linear hashing

Grows the number of buckets **one at a time** in a fixed order using a **split pointer**, so **no directory** is needed. When a split condition occurs (overflow or load factor exceeded) the bucket at the split pointer is split (not necessarily the overflowing one) using the next hash function $h_{level+1}$; overflow chains are used temporarily.

	Static	Extendible	Linear
Number of buckets	Fixed	Grows by splitting; directory doubles	Grows one bucket at a time
Directory	No	Yes	No
Overflow pages	Yes, many	No (almost)	Yes, temporary
Best for	Stable data size	Fast growth, uniform	Smooth growth

	B+ tree index	Hash index
Equality search	Good, O(log N)	Best, about O(1)
Range search	Excellent (linked leaves)	Not supported
Sorted order output	Yes	No
Used for	Most indexes, primary keys, ORDER BY	Exact-match lookups

```
CREATE INDEX idx_name ON STUDENT (Name);              -- B+ tree (default in most DBMS)
CREATE UNIQUE INDEX idx_email ON STUDENT (Email);
CREATE INDEX idx_dept_age ON STUDENT (Dept_id, Age);   -- composite index
DROP INDEX idx_name;                                   -- (MySQL: DROP INDEX idx_name ON STUDENT;)
```

Quick Revision - Unit V

- Transaction = logical unit of work. ACID: Atomicity, Consistency, Isolation, Durability.
- States: active -> partially committed -> committed; active/partially committed -> failed -> aborted -> terminated.
- Concurrency problems: lost update, dirty read, unrepeatable read, phantom.
- Serializable = equivalent to a serial schedule. Conflict test = precedence graph (cycle => not serializable).
- Recoverable, cascadeless, strict schedules. Isolation levels: read uncommitted ... serializable.
- Locks: S and X; 2PL (growing + shrinking), strict 2PL, rigorous 2PL.
- Timestamp ordering: R-TS, W-TS, rollback when order violated; Thomas write rule. Optimistic: read, validate, write.
- Deadlock: wait-for graph cycle. Prevention: wait-die, wound-wait. Recovery: victim selection, rollback.
- Failures: transaction, system crash, disk. Storage: volatile, non-volatile, stable.
- Log-based recovery, WAL, deferred / immediate update, checkpoint, shadow paging. ARIES: analysis, redo, undo.
- B+ tree: balanced, data in linked leaves, split (copy-up) on insert, borrow / merge on delete.
- Hashing: static, extendible (directory, global / local depth), linear (split pointer); no range queries.

Possible exam questions

1. Explain ACID properties and transaction states with a diagram.
2. Explain the problems caused by concurrent execution with examples.
3. What is serializability? Explain conflict and view serializability and test it using a precedence graph.
4. Explain recoverable, cascadeless and strict schedules; isolation levels.
5. Explain the two-phase locking protocol and its variants.
6. Explain timestamp-based and optimistic (validation) protocols.
7. What is deadlock? Explain detection, prevention (wait-die / wound-wait) and recovery.
8. Explain failure classification, log-based recovery, checkpoints and the ARIES algorithm.
9. Explain B+ tree structure and the insertion / deletion operations with examples.
10. Explain static, extendible and linear hashing.